

Redes Generativas Adversarias (GAN's)

29 de octubre de 2025

Índice general

1. Redes Generativas Adversarias (GAN's)	2
--	---

Capítulo 1

Redes Generativas Adversarias (GAN's)

Generador y Discriminador

Una Red Generativa Adversaria (GAN) se compone de dos redes neuronales entrenadas de forma enfrentada: un Generador (G) y un Discriminador (D).

El generador toma como entrada un vector aleatorio z (también llamado ruido latente), usualmente muestreado de una distribución simple p_z (por ejemplo, uniforme o normal estándar). A través de su arquitectura (típicamente una red neuronal profunda), G aplica una transformación determinista $G(z; \theta_G)$ parametrizada por θ_G para producir un ejemplo sintético $x_{\text{fake}} = G(z)$ en el espacio de datos (por ejemplo, una imagen). Por su parte, el discriminador $D(x; \theta_D)$ recibe como entrada un dato (ya sea real del conjunto de entrenamiento o falso producido por G) y emite una salida escalar $D(x)$ que interpreta como la probabilidad de que x provenga de la distribución real de datos p_{datos} en lugar de ser generado.

En términos formales, $D : \mathcal{X} \rightarrow [0, 1]$ es una función (aproximada por una red neuronal) que estima $D(x) \approx \Pr(\text{"dato real"}|x)$, mientras que $G : \mathcal{Z} \rightarrow \mathcal{X}$ produce muestras x nuevas intentando imitar p_{datos} .

El generador G recibe un vector $z \sim p_z$ desde el espacio latente y produce una muestra sintética $x_{\text{fake}} = G(z)$. A su vez, el discriminador D recibe ejemplos tanto del conjunto de datos reales $x_{\text{real}} \sim p_{\text{datos}}$ como ejemplos falsos x_{fake} provenientes de G . La tarea de D es emitir $D(x) \approx 1$ si x es real y $D(x) \approx 0$ si x es generado, actuando esencialmente como un clasificador binario. Por otro lado, G intenta engañar a D , produciendo datos sintéticos que D clasifique como reales (es decir, desea que $D(G(z)) \approx 1$). En síntesis, D y G están inmersos en un juego de suma cero donde compiten con objetivos opuestos: D busca minimizar su error de clasificación (distinguir real vs. falso correctamente), mientras G busca maximizar el error de D (que D se equivoque y marque sus falsos como reales).

Arquitectura del Generador y Discriminador

En la práctica, G y D suelen ser redes neuronales profundas con arquitecturas complementarias. El generador normalmente usa capas densas y/o convolucionales traspuestas (también llamadas de deconvolución) para transformar el vector z de dimensión d_z en una muestra x de la misma dimensión que los datos (por ejemplo, una imagen de

tamaño $64 \times 64 \times 3$). La capa de salida de G suele usar una función de activación *tanh* o sigmoide para acotar los valores al rango de datos válidos (por ejemplo, $[0, 1]$ para píxeles normalizados). Por otro lado, el discriminador suele ser una red similar a un clasificador binario: emplea capas convolucionales/densas para extraer características de x y finalmente una activación sigmoide $D(x) \in [0, 1]$ que indica la probabilidad estimada de que x sea real.

Es común que D use activaciones Leaky ReLU en sus capas ocultas (en lugar de ReLU estándar) para evitar la “muerte” de gradientes, y que se aplique normalización por lotes (batch normalization) en ambas redes para estabilizar el entrenamiento. Tanto G como D se entrenan simultáneamente mediante descenso de gradiente estocástico, pero con objetivos opuestos, como se detalla a continuación.

Entrenamiento adversarial como problema min-max

Formalmente, definimos una función de valor (o función objetivo) $V(D, G)$ que cuantifica la utilidad del discriminador menos la del generador. La formulación clásica propuesta por Goodfellow et al. (2014) es:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{datos}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] , \quad (1.1)$$

donde el discriminador D busca maximizar $V(D, G)$ y el generador G busca minimizarlo.

Intuitivamente, el primer término $\mathbb{E}_{x \sim p_{\text{datos}}} [\log D(x)]$ es la recompensa esperada que obtiene D por asignar alta probabilidad a los datos reales (quiere $\log D(x)$ lo más grande posible, i.e. $D(x) \rightarrow 1$ para x real). El segundo término $\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$ es la recompensa esperada por identificar correctamente las muestras falsas de G (desea $D(G(z)) \rightarrow 0$ para maximizar $\log(1 - D(G(z)))$). Por tanto, D trata de maximizar ambos términos: salir bien tanto en datos reales como en falsos. En cambio, el generador G aparece en el segundo término dentro de $D(G(z))$ y su objetivo es minimizar $V(D, G)$; es decir, G intentará hacer grande $\mathbb{E}_z [\log(1 - D(G(z)))]$ a la inversa, lo que implica forzar $D(G(z)) \rightarrow 1$ (que D se “confunda” otorgando alta probabilidad de real a datos falsos). En resumen, G minimiza la utilidad del discriminador, y D la maximiza, formalizando el concepto de entrenamiento adversarial en un marco de teoría de juegos de suma cero.

Deducción de la función objetivo y conexión con la divergencia Jensen–Shannon

La formulación de la Ecuación (1.1) es especialmente interesante porque, bajo ciertas condiciones, corresponde a minimizar una medida de discrepancia bien definida entre distribuciones de probabilidad. En particular, Goodfellow et al. demostraron que el entrenamiento adversarial original minimiza implícitamente la divergencia de Jensen–Shannon (JSD) entre la distribución real $p_{\text{datos}}(x)$ y la distribución generada $p_G(x)$. Para ver esto de forma matemática, analizamos el valor óptimo de la función objetivo para un generador fijo. Supongamos que G está fijo, definamos $p_G(x)$ como la distribución inducida por $G(z)$ cuando $z \sim p_z$. El discriminador $D(x)$, para un G dado, debe resolver:

$$D^*(x) = \arg \max_D \mathbb{E}_{x \sim p_{\text{datos}}} [\log D(x)] + \mathbb{E}_{x \sim p_G} [\log(1 - D(x))] . \quad (1.2)$$

Este máximo puede encontrarse optimizando $D(x)$ punto a punto (considerando x fijo).

Observemos que la expresión a maximizar es, para cada x , $p_{\text{datos}}(x) \log D(x) + p_G(x) \log(1 - D(x))$. Esta es una función concave en $D(x)$ (es entropía cruzada negativa de una Bernoulli) y podemos hallar su óptimo derivando con respecto a $D(x)$. Igualando a cero la derivada:

$$D^*(x) = \frac{p_{\text{datos}}(x)}{p_{\text{datos}}(x) + p_G(x)}, \quad (1.3)$$

es decir, el mejor discriminador posible para un G dado estima la probabilidad a posteriori de que x sea real mediante la razón de verosimilitud entre ambas distribuciones en.wikipedia.org . Este resultado tiene sentido intuitivo: si en un punto x la densidad de datos reales $p_{\text{datos}}(x)$ es mucho mayor que la de falsos $p_G(x)$, la mejor decisión de D es dar alta probabilidad a “real” (cercana a 1); por el contrario, si $p_G(x)$ supera a $p_{\text{datos}}(x)$ en ese x , lo óptimo es $D^*(x)$ cercano a 0 (clasificar como falso).

Ahora, sustituyendo $D^*(x)$ de la Ecuación (1.3) en la función objetivo, podemos calcular el valor óptimo de $V(D, G)$ para un generador dado. Denotemos $C(G) = V(D^*, G)$ este valor (considerando D óptimo en función de G). Entonces:

$$C(G) = E_{x \sim p_{\text{datos}}}[\log D^*(x)] + E_{x \sim p_G}[\log(1 - D^*(x))] \quad (1.4)$$

Sustituyendo $D^*(x) = \frac{p_{\text{datos}}(x)}{p_{\text{datos}}(x) + p_G(x)}$, obtenemos para cada término:

- Para datos reales: $\log D^*(x) = \log \frac{p_{\text{datos}}(x)}{p_{\text{datos}}(x) + p_G(x)} = \log p_{\text{datos}}(x) - \log \left(\frac{p_{\text{datos}}(x) + p_G(x)}{1} \right)$.
- Para datos generados: $\log(1 - D^*(x)) = \log \frac{p_G(x)}{p_{\text{datos}}(x) + p_G(x)} = \log p_G(x) - \log \left(\frac{p_{\text{datos}}(x) + p_G(x)}{1} \right)$.

Tomando las expectativas y sumando, se simplifica a:

$$\begin{aligned} C(G) &= E_{x \sim p_{\text{datos}}}[\log p_{\text{datos}}(x)] + E_{x \sim p_G}[\log p_G(x)] - E_{x \sim p_{\text{datos}}}[\log(p_{\text{datos}} + p_G)] \\ &= -\log(4) + 2D_{JS}(p_{\text{datos}} || p_G) \end{aligned} \quad (1.5)$$

donde D_{JS} es la divergencia de Jensen–Shannon. En la última igualdad se ha utilizado la definición de JSD en términos de las divergencias de Kullback–Leibler (KL) hacia la media $\frac{p_{\text{datos}} + p_G}{2}$:

$$D_{JS}(p_{\text{datos}} || p_G) = \frac{1}{2} D_{KL}(p_{\text{datos}} || \frac{p_{\text{datos}} + p_G}{2}) + \frac{1}{2} D_{KL}(p_G || \frac{p_{\text{datos}} + p_G}{2}) \quad (1.6)$$

lo que lleva precisamente a $C(G) = -\log 4 + 2, D_{JS}(p_{\text{datos}} || p_G)$. Observamos entonces que, a medida que el generador mejora haciendo p_G cercano a p_{datos} , la divergencia D_{JS} disminuye y por tanto $C(G)$ disminuye (recordando que $-\log 4$ es constante). En el óptimo global, $p_G = p_{\text{datos}}$ y $D_{JS} = 0$, obteniendo $C(G_{\text{óptimo}}) = -\log 4$. Este es el valor mínimo alcanzable de la función objetivo, que corresponde al equilibrio ideal donde las distribuciones coinciden y D falla aleatoriamente ($D(x) = 0.5$ para todo x). En conclusión, minimizar la función objetivo adversaria equivale a minimizar $D_{JS}(p_{\text{datos}} || p_G)$, de ahí que las GAN

originales se interpreten como un método de estimación generativa basado en la divergencia Jensen–Shannon.

Es útil destacar que la divergencia JS es una versión simétrica y suavizada de la divergencia KL. A diferencia de la KL estándar, la JSD está siempre bien definida y acotada (entre 0 y $\ln 2$ nats) incluso si las distribuciones no tienen apoyo común. Esto evita algunos problemas de las divergencias KL unilaterales, que pueden ser infinitas si $p_G(x) = 0$ en una región donde $p_{\text{datos}}(x) > 0$. En el contexto de GANs, esta propiedad contribuye a que el gradiente para mejorar G no desaparezca completamente incluso si al inicio p_G no cubre partes del soporte de p_{datos} . No obstante, en la práctica puede ocurrir desvanecimiento del gradiente para G cuando D es demasiado bueno, como veremos más adelante.

Derivadas de la entropía cruzada y retropropagación para D y G

La función objetivo adversaria de la Ecuación (1.1) puede entenderse como una suma de entropías cruzadas de clasificación binaria. En concreto, optimizar D equivale a ajustar un clasificador binario que asigna la etiqueta “real” (1) a muestras de p_{datos} y “falso” (0) a muestras de p_G . Si definimos por comodidad una pérdida (a minimizar) para el discriminador como el negativo de $V(D, G)$, obtenemos la función de pérdida de entropía cruzada binaria habitual:

$$L_D(\theta_D) = -\mathbb{E}_{x \sim p_{\text{datos}}}[\log D(x)] - \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] , \quad (1.7)$$

donde θ_D son los parámetros del discriminador. Esta es la misma forma de pérdida que se usa en regresión logística o en la capa softmax de un clasificador, aplicada a un batch mixto de ejemplos reales y falsos.

Gradiente para el discriminador (θ_D).

Dada la pérdida L_D en la Ecuación (1.7), la derivada respecto a la salida de D (antes de la sigmoide) es análoga a la de un clasificador binario. Para un ejemplo individual x con etiqueta $y = 1$ (real) o $y = 0$ (falso), la contribución a la pérdida es $-[y \log D(x) + (1 - y) \log(1 - D(x))]$. Derivando respecto a la entrada lineal $a = \logit(D(x))$ (es decir, $D(x) = \sigma(a)$ con σ la función sigmoide), se obtiene:

$$\frac{\partial L_p}{\partial a} = \sigma(a) - y = D(x) - y \quad (1.8)$$

que es exactamente el error de predicción (diferencia entre la salida $D(x)$ y la etiqueta y). Este resultado implica que al retropropagar, el gradiente que fluye hacia las capas internas de D es proporcional a $(D(x) - y)x$ (en el caso de una neurona lineal simple) o, en general, $(D(x) - y)$ multiplicado por las derivadas de las activaciones posteriores (según la regla de la cadena). Para un minibatch con ejemplos reales ($y = 1$) y falsos ($y = 0$), el gradiente descendente estocástico ajustará θ_D para disminuir este error en promedio. En términos más concretos, si $D(x_{\text{real}}) \approx 0$ (error, debería ser 1) entonces el gradiente será positivo empujando $D(x_{\text{real}})$ hacia arriba, y si $D(x_{\text{fake}}) \approx 1$ (error, debería ser 0) el gradiente será negativo empujando $D(x_{\text{fake}})$ hacia abajo. Así, D incrementa la probabilidad de real en muestras de p_{datos} y la reduce en muestras de p_G , tal como es deseable.

Gradiente para el generador (θ_G).

El generador no aparece explícitamente en L_D , pero influye a través de las muestras $x_{\text{fake}} = G(z)$ que alimentan a D . Su objetivo es minimizar la función adversaria, lo que equivale (en la formulación minimax original) a minimizar:

$$L_G(\theta_G) = -E_{z \sim p_z}[\log(1 - D(G(z)))] \quad (1.9)$$

es decir, G intenta engañar a D haciendo grande $D(G(z))$. Para obtener el gradiente de L_G respecto a θ_G , aplicamos la regla de la cadena diferenciando a través de D . Por simplicidad, consideremos un único z y denotemos $x_{\text{fake}} = G(z)$. La derivada de $\log(1 - D(G(z)))$ respecto a la entrada $a = \log(D(x_{\text{fake}}))$ del discriminador es:

$$\frac{\partial}{\partial a} \log(1 - D(x_{\text{fake}})) = -\frac{1}{1 - D(x_{\text{fake}})} * \sigma(a)(1 - \sigma(a)) = -\frac{D(x_{\text{fake}})(1 - D(x_{\text{fake}}))}{1 - D(x_{\text{fake}})} \quad (1.10)$$

o que se simplifica a $-D(x_{\text{fake}})$ (dado que $\sigma(a) = D(x)$). Aplicando nuevamente la regla de la cadena hacia x_{fake} , obtenemos:

$$\nabla x_{\text{fake}} L_G = -D(x_{\text{fake}}) \nabla D(x_{\text{fake}}) \quad (1.11)$$

y finalmente propagando el gradiente hasta los parámetros θ_G mediante $\nabla_{\theta_G} G(z)$:

$$\nabla_{\theta_G} L_G = -D(G(z)) \nabla_x D(x)|_{x=G(z)} \cdot \nabla_{\theta_G} G(z), \quad (1.12)$$

donde $\nabla_x D(x)|_{x=G(z)}$ es el gradiente del discriminador evaluado en la muestra sintética. Esta expresión describe cómo G ajusta sus parámetros: mediante el gradiente transmitido a través del discriminador, ponderado por $-D(G(z))$. Intuitivamente, si para la muestra generada x_{fake} el discriminador da una probabilidad alta $D(x_{\text{fake}}) \approx 1$ (lo clasificó como real, que es bueno para G), entonces el término $-D(G(z))$ será cercano a -1 y G actualizará sus parámetros ligeramente (incluso podría invertir el gradiente del discriminador si este estaba haciendo que x_{fake} se pareciera demasiado a un real). En cambio, si $D(G(z))$ es bajo (0), indicando que D detectó el engaño, $-D(G(z)) \approx 0$ y el generador recibirá poco gradiente para ese z (el daño ya está hecho: D está muy seguro de que es falso, generando la problemática de gradiente saturado). Este análisis refleja el problema de desvanecimiento de gradiente para G : cuando D es demasiado eficaz y $D(G(z)) \rightarrow 0$, el término de actualización de G se acerca a cero y le cuesta mejorar. Para mitigar esto, en la práctica se suele modificar el objetivo del generador.

Objetivo no saturante

En la implementación original de GANs, Goodfellow recomendó que, en lugar de usar la pérdida $L_G = -\mathbb{E}[\log(1 - D(G(z)))]$ (que puede saturarse), es mejor definir una pérdida alternativa para G :

En la implementación original de GANs, Goodfellow recomendó que, en lugar de usar la pérdida $L_G = -\mathbb{E}[\log(1 - D(G(z)))]$ (que puede saturarse), es mejor definir una pérdida alternativa para G :

$$L_G^{\text{NS}}(\theta_G) = -\mathbb{E}_z[\log D(G(z))] , \quad (1.13)$$

que equivale a maximizar $\mathbb{E}[\log D(G(z))]$. Este objetivo no saturante produce el mismo punto óptimo ($p_G = p_{\text{datos}}$) pero proporciona gradientes más fuertes al generador cuando $D(G(z))$ es pequeño. De hecho, derivando L_G^{NS} se obtiene:

$$\nabla_{\theta_G} L_G^{\text{NS}} = -\nabla_{\theta_G} E_z[\log D(G(z))] = -\frac{1}{D(G(z))} \nabla_{\theta_G} D(G(z)) \quad (1.14)$$

lo que lleva a un término $-\frac{1}{D(G(z))}$ en lugar de $-D(G(z))$ en la Ec. (1.12). Si $D(G(z))$ es muy pequeño (situación mala para G), ahora $-1/D(G(z))$ será un factor enorme, empujando fuertemente a G a mejorar esa muestra. Por ello, esta variante evita la saturación de gradiente y suele conducir a entrenamientos más estables en la práctica.

Ejemplo numérico de actualización de parámetros

Para ilustrar los cálculos anteriores, consideremos un ejemplo sencillo de una GAN en la que tanto D como G son modelos muy simples (lineales de una sola neurona) entrenados con un solo ejemplo real y uno generado. Aunque este escenario es artificial, ayudará a ver paso a paso cómo se actualizan los parámetros. Supondremos:

- El discriminador D es una neurona con activación sigmoide: $D(x) = \sigma(w_D * x + b_D)$, con parámetro w_D (peso) y b_D (sesgo).
- El generador G es también lineal: $G(z) = w_G * z + b_G$, con parámetro w_G (y consideraremos $b_G = 0$ por simplicidad).
- Tenemos un único dato real $x_{\text{real}} = 1.0$ (con etiqueta $y = 1$) y tomamos una muestra latente $z = 1.0$ para generar $x_{\text{fake}} = G(1) = w_G$. Inicializamos $w_D = 0$, $b_D = 0$, $w_G = 0$ al comienzo.
- Usamos la pérdida de entropía cruzada estándar para D y el objetivo no saturante para G . Emplearemos una tasa de aprendizaje $\alpha = 1.0$ (solo para simplificar los cálculos manuales; normalmente sería mucho menor).

Paso 1: Forward pass inicial. Con los pesos iniciales:

- $D(x_{\text{real}}) = \sigma(0 \cdot 1 + 0) = \sigma(0) = 0.5$.
- $D(x_{\text{fake}}) = D(G(1)) = D(w_G \cdot 1 + 0) = \sigma(0) = 0.5$ (ya que $w_G = 0$ inicialmente, $G(1) = 0$). El discriminador no tiene ninguna capacidad inicialmente y da 0.5 a ambos, como es de esperarse.

Paso 2: Cálculo de pérdidas:

- Pérdida del discriminador: $L_D = -[\log D(x_{\text{real}}) + \log(1 - D(x_{\text{fake}}))] = -[\log(0.5) + \log(1 - 0.5)] = -[\log 0.5 + \log 0.5] = -2 \log 0.5 = \ln(4) \approx 1.3863$.
- Pérdida del generador (no saturante): $L_G^{\text{NS}} = -\log D(x_{\text{fake}}) = -\log(0.5) = \ln(2) \approx 0.6931$.

Paso 3: Gradiente para D

Para D , computamos gradiente respecto a sus entradas:

- Para x_{real} : $D(x_{\text{real}}) = 0.5$ con $y = 1$, el error es $(0.5 - 1) = -0.5$.
- Para x_{fake} : $D(x_{\text{fake}}) = 0.5$ con $y = 0$, el error es $(0.5 - 0) = 0.5$. Estos errores se propagan a w_D y b_D . Derivadas:

$$\frac{\partial L_D}{\partial \omega_D} = (D(x_{\text{real}}) - 1)x_{\text{real}} + (D(x_{\text{fake}}) - 0)x_{\text{fake}} = (-0.5) * 1 + (0.5) * 0 = -0.5 \quad (1.15)$$

$$\frac{\partial L_D}{\partial b_D} = -0.5 + 0.5$$

Notamos que b_D no tiene gradiente (lo cual puede ocurrir aquí por simetría inicial).

Paso 4: Actualización de D

Usando descenso de gradiente:

- $w_D^{\text{nuevo}} = w_D - \alpha \frac{\partial L_D}{\partial w_D} = 0 - 1.0 \cdot (-0.5) = 0.5$.
- $b_D^{\text{nuevo}} = b_D - \alpha \frac{\partial L_D}{\partial b_D} = 0 - 1.0 \cdot 0 = 0$.

Ahora el discriminador tiene peso 0.5 y sesgo 0.

Paso 5: Gradientes para G

Con el discriminador actualizado, recalculamos $D(x_{\text{fake}})$ para la retropropagación de G :

- x_{fake} actual sigue siendo 0 (ya que w_G no cambió aún).
- $D(x_{\text{fake}}) = \sigma(0.5 \cdot 0 + 0) = \sigma(0) = 0.5$ (curiosamente sigue en 0.5 por casualidad en este paso). El gradiente para G usando $L_G^{\text{NS}} = -\log D(G(z))$ es:

$$\frac{\partial L_G^{\text{NS}}}{\partial \omega_G} = -\frac{1}{D(G(z))} \frac{\partial D(G(z))}{\partial \omega_G} \quad (1.16)$$

Ahora, $\frac{\partial D(G(z))}{\partial \omega_G}$ involucra derivar a través de D :

$$\frac{\partial D}{\partial x}|_{x=0} = D(0)[1 - D(0)] = 0.5 * 0.5 = 0.25 \quad (1.17)$$

y $\frac{\partial x_{\text{fake}}}{\partial \omega_G} = z = 1$. Por lo tanto:

$$\frac{\partial D(G(z))}{\partial \omega_G} = 0.25 * 1 = 0.25 \quad (1.18)$$

Finalmente:

$$\frac{\partial L_G^{\text{NS}}}{\partial \omega_G} = -\frac{1}{0.5}(0.25) = -0.5 \quad (1.19)$$

Paso 6: Actualización de G

Aplicando descenso de gradiente a w_G :

$$\omega_G^{nuevo} = \omega_G - \alpha \frac{\partial L_G^{NS}}{\partial \omega_G} = 0 - 1(-0.5) = 0.5 \quad (1.20)$$

Tras este primer ciclo de actualizaciones, tenemos $w_D = 0.5$, $b_D = 0$, $w_G = 0.5$.

Podemos inspeccionar cómo cambió el comportamiento:

- Antes, $D(1) = 0.5$ y $D(0) = 0.5$. Ahora, con $w_D = 0.5$,
- $D(x_{\text{real}} = 1) = \sigma(0.5 \cdot 1) \approx 0.62$ (ha subido la probabilidad de real para el dato real, como deseaba D).
- $D(x_{\text{fake}} = G(1) = 0.5) = \sigma(0.5 \cdot 0.5) = \sigma(0.25) \approx 0.56$. Inicialmente D daba 0.5 a este falso; ahora lo evalúa en 0.56, o sea está empezando a creerlo ligeramente más real, porque G logró aumentar un poco su calidad (antes $x_{\text{fake}} = 0$, ahora $x_{\text{fake}} = 0.5$, más cercano al real de 1).
- La pérdida de D habrá disminuido y la de G quizás aumentado ligeramente en este paso, pero idealmente en más iteraciones G seguirá aumentando w_G para producir x_{fake} más grande y acercarse a 1, mientras D ajustará w_D para mantener la separación. En un entrenamiento prolongado, x_{fake} tendería hacia 1 y D tendería a saturar en 0.5 (ya incapaz de distinguir).

Relación con autoencoders y variaciones: información mutua y codificación

Aunque las GANs surgieron independientemente de los autoencoders, conceptualmente comparten la idea de un espacio latente y un decodificador G . En un autoencoder tradicional, se entrena conjuntamente un encoder $E : X \rightarrow Z$ y un decoder $G : Z \rightarrow X$ para reconstruir los datos, lo que fuerza a Z a ser una representación compresada de X . En una GAN estándar no existe tal encoder inverso; sin embargo, algunas variantes conocidas como BiGAN o ALI (Adversarial Learned Inference) introducen un encoder entrenado adversarialmente para mapear $x \rightarrow z$ de modo que la pareja de encoder/decoder sea consistente (el encoder produce códigos cuya distribución coincide con p_z y se puede recuperar el dato con el generador). Estas variantes establecen un puente entre GANs y autoencoders, permitiendo inferencia de z a partir de x .

Otra conexión importante es con los Autoencoders Variacionales (VAE). Los VAE también aprenden un espacio latente y un decodificador generativo, pero lo hacen maximizando una cota inferior de la probabilidad de los datos (asumiendo un modelo generativo explícito) y usando una función de costo diferente que incluye un término de regularización KL sobre z .

A diferencia de GAN, en VAE el z obtenido tiende a cubrir una región densa del espacio latente (normal gaussiana), mientras que en GAN el apoyo de z efectivo podría fracturarse (ya que no se optimiza una probabilidad explícita). Este es un tema sutil: en GAN no hay

garantía de cobertura completa, pero en práctica con un buen entrenamiento se espera que $G(Z)$ abarque las variaciones de X .

Un concepto relevante introducido en algunas GANs es la información mutua entre partes de z y la salida x . Por ejemplo, la arquitectura InfoGAN (Chen et al., 2016) añadió a la pérdida un término que maximiza la información mutua $I(c; G(c, z))$ entre ciertos códigos latentes c (una parte de z separada) y la muestra generada.

Esto obliga a G a no “olvidar” esos componentes latentes, haciendo que tengan efecto directo en x (por ejemplo, c podrían ser variables categóricas o continuas que InfoGAN trata de asociar a atributos como dígito escrito, rotación, etc.). Maximizar la información mutua $I(c; x)$ equivale a minimizar la incertidumbre de c dado x (es decir, que a partir de una imagen generada puedas inferir el valor de c). InfoGAN logra este máximo agregando un segundo cabezal al discriminador que estima la probabilidad de c dado x , y añadiendo una pérdida de clasificación correspondiente, tal que en el equilibrio se demuestra que se maximiza un límite inferior de $I(c; x)$. Este es un ejemplo de cómo ideas de codificación de información pueden integrarse en la GAN para mejorar la interpretabilidad del espacio latente.

En general, las GANs puras no maximizan explícitamente ninguna función de probabilidad como hacen los modelos de estimación densidad (p. ej. modelos de flujo, VAE), lo que dificulta evaluar cuánta de la información de p_{datos} han capturado. Métricas como $I(x; z)$ (información mutua entre x generado y semilla z) pueden dar pistas: idealmente $I(x; z) = H(z)$, es decir, toda la incertidumbre del vector latente se refleja en la salida (ninguna se perdió). Si $I(x; z)$ es menor, significa que diferentes z producen outputs parecidos (colapso parcial). Aunque calcular esto no es trivial sin un encoder, se pueden entrenar aproximaciones o suponer invertibilidad local de G . De cualquier manera, incluir objetivos de información (como InfoGAN) o utilizar modelos híbridos que combinen un encoder (como los mencionados ALI/BiGAN) puede ayudar a hacer el modelo generativo más invertible y con un espacio latente más significativo.

La información mutua en regularización

Otra variante interesante es introducir ruido en x real de manera controlada y forzar a D a predecir algo sobre la instancia limpia, lo cual está relacionado con maximizar información compartida entre entrada y salida bajo ciertas transformaciones (técnicas usadas en GANs condicionales o de datos aumentados). Sin alejarnos demasiado, vale la pena recalcar que la flexibilidad de la formulación adversarial permite incorporar términos adicionales a la función objetivo para lograr propiedades deseadas (sea invariancia a ciertas transformaciones, correspondencia latente-datos, incorporación de etiquetas en GANs condicionales, etc.), siempre diseñando D para que evalúe correctamente esas condiciones.